

Claude Code

Best Practice Rehberi

Command · Skill · Agent · Orchestration

Kaynak: github.com/shanraishan/claude-code-best-practice
Boris Cherny, Thariq Shihpar ve Claude Code Ekibi tavsiyeleri
Anthropic Resmi Dokümantasyonu baz alınarak derlenmiştir

Mart 2026 · v2.1+

İçindekiler

1. Temel Kavramlar

1.1 Command (Komut)

1.2 Skill (Beceri)

1.3 Agent (Alt Ajan)

1.4 Karşılaştırma Tablosu

2. Hiyerarşi ve Karar Akışı

2.1 Günlük Kullanım Dağılımı

2.2 Karar Ağacı

3. Orchestration Workflow

3.1 Command → Agent → Skill Pattern

3.2 Çalışan Örnek

4. Memory (CLAUDE.md) Yönetimi

4.1 Yükleme Mekanizması

4.2 Monorepo Stratejisi

5. MCP Server Konfigürasyonu

6. Settings ve Konfigürasyon

7. Hooks (Kancalar)

8. Agent Memory (Kalıcı Hafıza)

9. Altın Kurallar ve İpuçları

9.1 Workflow İpuçları

9.2 Context Yönetimi

9.3 Debugging

9.4 Araçlar ve Yardımcılar

10. Yapılması ve Yapılmaması Gerekenler

1. Temel Kavramlar

Claude Code CLI üç temel yapı taşı üzerine kuruludur. Her birinin farklı bir görevi, çalışma şekli ve context maliyeti vardır.

1.1 Command (Komut)

Tanım: Kullanıcının `/komut-adi` yazarak tetiklediği giriş noktası prompt şablonları. Mevcut context'e enjekte olur, yeni izole context açmaz.

Konum: `.claude/commands/<isim>.md`

Frontmatter Alanları

Alan	Tip	Zorunlu	Açıklama
<code>description</code>	string	Önerilen	Komutun açıklaması. Autocomplete'de görünür
<code>argument-hint</code>	string	Hayır	Argüman ipucu (ör: <code>[dosya-adi]</code>)
<code>allowed-tools</code>	string	Hayır	İzin istemeden kullanılacak araçlar
<code>model</code>	string	Hayır	Kullanılacak model: <code>haiku</code> , <code>sonnet</code> , <code>opus</code>

Dinamik Değişkenler

Değişken	Açıklama
<code>\$ARGUMENTS</code>	Komuta geçilen tüm argümanlar
<code>\$0</code> , <code>\$1</code> , <code>\$2</code>	0-indexed argüman erişimi
<code>\${CLAUDE_SESSION_ID}</code>	Aktif oturum kimliği
<code>!`shell-komutu`</code>	Shell çıktısını prompt'a enjekte eder (çalışma anında)

Örnek: Basit Komut

```
---
description: Firmware'i derle ve test et
model: haiku
---

Projedeki firmware'i derle ve birim testleri çalıştır.
Sonuçları özetle.
```

Örnek: Tam Donanımlı Komut

```
---
description: GitHub issue'yu numarasıyla çöz
argument-hint: [issue-numarasi]
allowed-tools: Read, Edit, Write, Bash(gh *), Bash(npm test *)
model: sonnet
---

GitHub issue $0'ı kodlama standartlarımıza göre çöz.

## Bağlam
- PR diff: !`gh pr diff`
- Issue detayı: !`gh issue view $0`

## Adımlar
1. Issue açıklamasını oku
2. Gereksinimleri anla
3. Düzeltmeyi uygula
4. Testleri yaz
5. Commit oluştur

Oturum: ${CLAUDE_SESSION_ID}
```

Öncelik Sırası

Konum	Kapsam	Öncelik
<code>.claude/commands/</code>	Sadece bu proje	1 (En yüksek)
<code>~/.claude/commands/</code>	Tüm projeler	2
<code><plugin>/commands/</code>	Plugin etkinse	3 (En düşük)

1.2 Skill (Beceri)

Tanım: Yeniden kullanılabilir bilgi paketleri ve iş akışları. Mevcut context'e enjekte olur. İhtiyaç duyulduğunda yüklenir (progressive disclosure).

Konum: `.claude/skills/<isim>/SKILL.md`

Frontmatter Alanları

Alan	Açıklama
<code>name</code>	Görünen ad ve /slash-command tanımlayıcı. Yazılmazsa izin adı kullanılır
<code>description</code>	Ne yaptığı. Autocomplete ve auto-discovery için
<code>user-invocable</code>	<code>false</code> yapılırsa / menüsünden gizlenir — sadece agent preloading için
<code>allowed-tools</code>	Skill aktifken izin istemeden kullanılacak araçlar

İki Farklı Skill Pattern'i

PATTERN 1: AGENT SKILL (PRELOADED)

Agent'in `skills:` alanında tanımlanır. Agent başlarken context'e otomatik enjekte olur. Agent'ın "bildiği şeyler" kategorisindedir.

PATTERN 2: STANDALONE SKILL (ON-DEMAND)

`/skill-adi` yazarak veya `Skill()` tool'u ile çağrılır. Bağımsız çalışır, hiçbir agent'a bağlı değildir. İhtiyaç olduğunda yüklenir.

Örnek: Standalone Skill

```
---
name: svg-card-creator
description: >
  Veri ile SVG görsel kart oluşturur. Rapor veya
  dashboard çıktısı gerektiğinde kullan.
user-invocable: true
---
```

SVG Kart Oluşturucu

Kurallar

- Boyut: 800x400px, viewBox kullan
- Font: sistem fontu, embed etme
- Renkler: CSS değişkeni ile tanımla
- Çıktı: output/ dizinine yaz

Şablon Yapısı

- Başlık: üst 60px, koyu arka plan
- İçerik: orta alan, grid düzeni
- Alt bilgi: tarih ve versiyon

Örnek: Agent Skill (Preloaded — Gizli)

```
---
name: api-conventions
description: API endpoint yazım kuralları
user-invocable: false
---
```

API Kuralları

- REST endpoint: /api/v1/{resource}
- Hata formatı: { error: { code, message } }
- Pagination: cursor-based
- Auth: Bearer token header'da

1.3 Agent (Alt Ajan)

Tanım: Kendi izole context'inde çalışan otonom aktör. Kendine özel araçlar, izinler, model, hafıza ve kimlik sahibidir.

Konum: `.claude/agents/<isim>.md`

Frontmatter Alanları

Alan	Tip	Açıklama
<code>name</code>	string	Görünen ad
<code>description</code>	string	Ne zaman çağrılacağı. "PROACTIVELY" yazılırsa Claude otomatik çağırır
<code>tools</code>	string	İzin verilen araçlar: Read, Write, Edit, Bash . Yazılmazsa tümü inherit
<code>disallowedTools</code>	string	Yasaklı araçlar
<code>model</code>	string	haiku, sonnet, opus, inherit
<code>permissionMode</code>	string	acceptEdits, plan, bypassPermissions
<code>maxTurns</code>	int	Maksimum tur sayısı
<code>skills</code>	list	Başlangıçta preload edilecek skill'ler
<code>mcpServers</code>	list	Agent'a özel MCP server'lar
<code>hooks</code>	object	Lifecycle hook'ları (PreToolUse, PostToolUse, Stop)
<code>memory</code>	string	Kalıcı hafıza: user, project, local
<code>background</code>	bool	true → arka planda çalışır
<code>color</code>	string	CLI çıktı rengi (green, magenta vb.)

Örnek: Code Reviewer Agent

```

---
name: code-reviewer
description: >
  Kod yazıldıktan sonra kalite, güvenlik ve bakım
  standartlarını kontrol eder. PROACTIVELY çağrılabilir.
tools: Read, Write, Edit, Bash
model: sonnet
memory: user
color: green
---
```

Sen bir kod incelemecisisin. Kodu incelerken keşfettiğin pattern'leri, kuralları ve tekrarlayan sorunları agent hafızana kaydet.

```

## İnceleme Kontrol Listesi
1. Güvenlik: Hardcoded secret, injection, XSS
2. Performans: N+1 query, gereksiz hesaplama
3. Test: Yeterli kapsam, edge case'ler
4. Stil: Proje kurallarına uygunluk
```

⚠ KRİTİK KURAL: AGENT'LAR ARASI İLETİŞİM

Agent'lar bash komutuyla birbirini **çağırılmaz**. Bunun yerine **Agent()** tool'unu kullan:

```

Agent(
  subagent_type="code-reviewer",
  description="Kodu incele",
  prompt="Son commit'i güvenlik açısından incele",
  model="sonnet"
)
```

"launch", "run" gibi belirsiz kelimeler kullanma — Claude bash komutu olarak yorumlayabilir.

1.4 Karşılaştırma Tablosu

Özellik	Command	Skill	Agent
Context	Mevcut context'e enjekte	Mevcut context'e enjekte	Yeni izole context
Tetikleme	<code>/komut-adi</code>	<code>/skill-adi</code> veya preload	<code>Agent()</code> tool
Amaç	Orkestrasyon, kullanıcı etkileşimi	Bilgi / know-how sağlama	Bağımsız görev yürütme
Context maliyeti	Düşük	Orta	Yüksek
Kendi modeli	Evet	Hayır	Evet
Kalıcı hafıza	Hayır	Hayır	Evet
Araç kısıtlama	allowed-tools	Hayır	tools / disallowedTools
Ne zaman	Her iş akışının başlangıcı	Tekrarlayan domain bilgisi	İzole, ağır görevler

2. Hiyerarşi ve Karar Akışı

2.1 Günlük Kullanım Dağılımı



ALTIN KURAL

"Vanilla Claude Code, küçük görevlerde herhangi bir workflow'dan daha iyidir." Gereksiz karmaşıklık ekleme. Workflow sadece tekrarlayan veya koordinasyon gerektiren işler için.

2.2 Karar Ağacı

```

Görev basit mi?
→ Evet → Vanilla Claude Code, hiçbir şey kurma
→ Hayır ↓

Tekrarlayan bir iş akışı mı?
→ Evet → COMMAND yaz (/build, /test, /deploy)
→ Hayır ↓

Birden fazla yerde kullanılan domain bilgisi mi?
→ Evet → SKILL yaz (protocol-spec, coding-patterns)
→ Hayır ↓

Ana context'i kirletmeden izole iş mi?
→ Evet → AGENT tanımla (reviewer, test-runner)
→ Hayır → Vanilla Claude Code
  
```

3. Orchestration Workflow

3.1 Command → Agent → Skill Pattern

En güçlü kullanım şekli üç yapıyı katmanlı birleştirmektir:

Katman	Rol	Açıklama
Command	Giriş noktası	Kullanıcıyla etkileşir, agent ve skill'leri koordine eder
Agent	Veri çekici	Preloaded skill ile veri toplar, sonucu command'a döner
Skill	Çıktı üretici	Bağımsız çalışır, agent'tan gelen veriyle çıktı üretir

3.2 Çalışan Örnek: Weather Orchestrator

```
# Akış
Kullanıcı → /weather-orchestrator (Command)
├─ Adım 1: AskUser → Celsius mı Fahrenheit mı?
├─ Adım 2: Agent tool → weather-agent
│   └─ Preloaded Skill: weather-fetcher
│       └─ Open-Meteo API'den sıcaklık çeker
│           └─ Döndürür: temperature=26, unit=Celsius
├─ Adım 3: Skill tool → weather-svg-creator
│   └─ Oluşturur: weather.svg
│       └─ Yazar: output.md
└─ Çıktı: SVG kart + özet dosya
```

Command Dosyası

```
# .claude/commands/weather-orchestrator.md
---
description: Hava durumu verisi çek ve SVG kart oluştur
model: haiku
---

1. Kullanıcıya sor: Celsius mi Fahrenheit mi?
2. weather-agent'ı çağır (sıcaklık çekmesi için)
3. Dönen veriyi /weather-svg-creator skill'ine ver
4. Oluşturulan SVG ve özeti kullanıcıya sun
```

Agent Dosyası (Preloaded Skill ile)

```
# .claude/agents/weather-agent.md
---
name: weather-agent
description: Hava durumu verisi çeken ajan
tools: Bash(curl *)
model: haiku
skills:
  - weather-fetcher # Başlangıçta preload edilir
---

Preloaded skill'deki talimatlara göre sıcaklık verisini çek
ve sonucu döndür.
```

Temel Prensipler

- **Command** = orkestratör (kullanıcı etkileşimi + koordinasyon)
- **Agent** = veri çekme (preloaded skill ile)
- **Skill** = bağımsız çıktı üretme
- Her bileşenin **tek bir sorumluluğu** var: Fetch (agent) → Render (skill)

4. Memory (CLAUDE.md) Yönetimi

4.1 Yükleme Mekanizması

Tip	Yükleme	Açıklama
Ancestor (üst dizinler)	Başlangıçta otomatik	Claude yukarı yürür, tüm CLAUDE.md'leri yükler
Descendant (alt dizinler)	Tembel (lazy)	Sadece o dizindeki dosyalarla etkileşimde yüklenir
Sibling (kardeş dizinler)	ASLA	frontend/ çalışırken backend/CLAUDE.md yüklenmez
Global	Her zaman	~/ .claude/CLAUDE.md — tüm projelerde geçerli

4.2 Monorepo Stratejisi

```
/proje-root/
├── CLAUDE.md # Root – ortak kurallar (her zaman yüklenir)
├── frontend/
│   └── CLAUDE.md # Frontend kuralları (lazy)
├── backend/
│   └── CLAUDE.md # Backend kuralları (lazy)
├── firmware/
│   └── CLAUDE.md # Firmware kuralları (lazy)
└── api/
    └── CLAUDE.md # API kuralları (lazy)
```

CLAUDE.MD KURALLARI

- Dosya başına **200 satırı geçme** (ideal: 60 satır)
- `CLAUDE.local.md` ile kişisel tercihleri ayır (`.gitignore` 'a ekle)
- `/memory` ve `/rules` komutları hiçbir şeyi **garanti etmez** — MUST yazmak bile yetmez
- Bilgiyi CLAUDE.md'ye tıkmak yerine **skill'lere** böl

5. MCP Server Konfigürasyonu

MCP (Model Context Protocol) server'lar Claude Code'u harici araçlara, veritabanlarına ve API'lere bağlar.

Konfigürasyon Konumları ve Öncelikleri

Konum	Kapsam	Öncelik
Agent frontmatter (<code>mcpServers</code>)	O agent'a özel	1 (En yüksek)
<code>.mcp.json</code> (repo root)	Proje	2
<code>~/.claude.json</code>	Kullanıcı (tüm projeler)	3

Önerilen MCP Server'lar

Server	Paket	İşlev
Context7	<code>@upstash/context7-mcp</code>	Güncel kütüphane dokümantasyonunu çeker, hallucination'ı önler
Playwright	<code>@playwright/mcp</code>	Browser otomasyonu — UI test, screenshot, form
Chrome DevTools	—	Gerçek Chrome'a bağlanır — console, network, DOM debug
DeepWiki	<code>deepwiki-mcp</code>	GitHub repo'lar için wiki-style dokümantasyon
Excalidraw	—	Mimari diyagram, flowchart oluşturma

Örnek `.mcp.json`

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp"]
    },
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp"]
    }
  }
}
```

UYARI: FAZLA MCP = FAZLA KARMAŞIKLIK

"15 MCP server kurdum, daha fazla = daha iyi diye düşünerek. Sonuçta günlük sadece 4 tanesini kullanıyordum." — r/mcp (682 upvote). Sadece gerçekten kullandıklarını ekle.

6. Settings ve Konfigürasyon

Dosya Hiyerarşisi

Dosya	Kapsam	Açıklama
<code>~/.claude/settings.json</code>	Global	Tüm projelerde geçerli kullanıcı ayarları
<code>.claude/settings.json</code>	Proje	Takım paylaşımlı proje ayarları
<code>.claude/settings.local.json</code>	Kişisel	git-ignored kişisel overrides
<code>managed-settings.json</code>	Organizasyon	Override edilemez, en yüksek öncelik

Önerilen Ayarlar

- **Thinking mode:** `true` — Claude'un reasoning sürecini görmek için
- **Output Style:** `Explanatory` — ★ Insight kutuları ile detaylı çıktı (yeni codebase'de)
- **Output Style:** `Learning` — koçluk modu (öğrenme amaçlı)
- `settings.json` 'u **git'e check-in et** — takım faydalansın
- `env` alanını kullan — wrapper script'lere gerek kalmaz (84 ortam değişkeni)

7. Hooks (Kancalar)

Agentic döngü **dışında**, belirli event'lerde çalışan deterministik scriptler.

Hook Event'leri

Event	Ne Zaman Tetiklenir
<code>PreToolUse</code>	Araç çağrılmadan hemen önce
<code>PostToolUse</code>	Araç çağrısı tamamlandıktan sonra
<code>UserPromptSubmit</code>	Kullanıcı prompt gönderdiğinde
<code>Stop</code>	Claude durduğunda
<code>SubagentStart</code> / <code>SubagentStop</code>	Agent başlatıldığında / durdurulduğunda
<code>Notification</code>	Bildirim geldiğinde
<code>PreCompact</code>	Compact işleminden önce
<code>SessionStart</code> / <code>SessionEnd</code>	Oturum açılış / kapanış

Kullanım Alanları

- Git commit olduğunda ses çalma
- İzin isteklerini Slack'e yönlendirme
- Tool çağrılarını loglama
- Claude durduğunda devam etmesini sağlama (nudge)
- Her işlem sonrası lint çalıştırma

8. Agent Memory (Kalıcı Hafıza)

Claude Code v2.1.33'te tanıtıldı. Her agent'a kendi kalıcı hafızasını verir — oturumlar arası öğrenme sağlar.

Memory Scope'ları

Scope	Konum	Kimler Görür
<code>user</code>	<code>~/.claude/agent-memory/<agent>/</code>	Sadece sen, tüm projeler
<code>project</code>	<code>.claude/agent-memory/<agent>/</code>	Tüm takım, bu proje
<code>local</code>	<code>.claude/agent-memory.local/<agent>/</code>	Sadece sen, bu proje

Mekanizma

- Başlangıçta `MEMORY.md` 'nin ilk **200 satırı** system prompt'a enjekte edilir
- `Read` , `Write` , `Edit` otomatik etkinleştirilir

- 200 satırı geçerse agent detayları topic-specific dosyalara taşır
- Skills = statik bilgi (başlangıçta) ↔ Memory = dinamik bilgi (zamanla birikir)

Örnek: Hafızalı Agent

```
---
name: api-developer
tools: Read, Write, Edit, Bash
model: sonnet
memory: project
skills:
  - api-conventions
  - error-handling-patterns
---
```

API endpoint'leri implement et. Preloaded skill'lerdeki kurallara uy. Çalışırken mimari kararları ve keşfettiğin pattern'leri hafızana kaydet.

Başlamadan önce hafızanı kontrol et.

```
~/claude/agent-memory/api-developer/
├─ MEMORY.md # Ana dosya (ilk 200 satır yüklenir)
├─ rest-patterns.md # Konu-spesifik
└─ error-catalog.md # Konu-spesifik
```

9. Altın Kurallar ve İpuçları

9.1 Workflow İpuçları

Kural	Açıklama
Her zaman plan mode ile başla	<code>/plan</code> komutu — önce oku, sonra uygula
<code>ultrathink</code> kullan	Prompt'a "ultrathink" ekle → yüksek effort reasoning
Feature-specific agent tanımla	"QA engineer" yerine "auth-module-reviewer" gibi spesifik
Skills ile progressive disclosure	Tüm bilgiyi CLAUDE.md'ye tıkma, skill'lere böl
Faz bazlı plan yap	Her fazda birim + entegrasyon + otomasyon testi olsun
Cross-model review kullan	Planı başka model ile review ettir (ör: Codex)
Claude'a röportaj yaptır	<code>AskUserQuestion</code> tool ile spec oluştur, sonra yeni oturumda execute et
Detaylı spec yaz	Belirsizliği azalt — ne kadar spesifiksen o kadar iyi çıktı

9.2 Context Yönetimi

Kural	Açıklama
%50'de /compact yap	"Agent dumb zone"dan kaçın — context dolmadan sıkıştır
<code>/clear</code> ile sıfırla	Görev değiştiğinde oturumu sıfırla
<code>/context</code> ile takip et	Context kullanımını renkli grid olarak gör
<code>Esc Esc</code> veya <code>/rewind</code>	Claude yanlış yöne giderse düzeltme yerine geri sar
" <code>use subagents</code> " de	Ağır işi subagent'a offload et, ana context temiz kalsın
Oturumları adlandır	<code>/rename [TODO - refactor]</code> → sonra <code>/resume</code> ile devam

9.3 Debugging

Yöntem	Açıklama
<code>/doctor</code>	Claude Code kurulum sağlığını kontrol et
Background task olarak çalıştır	Log görmek istediğin terminal'i arka planda aç
MCP ile browser debug	Claude in Chrome / Playwright / Chrome DevTools MCP
Screenshot ver	Görsel sorunlarda ekran görüntüsü paylaş
Farklı model ile QA	İmplementasyonu başka model/araç ile review et
Claude'a meydan oku	"Bu değişiklikleri eleştir, testi geçmeden PR açma"

9.4 Araçlar ve Yardımcılar

Araç	Açıklama
iTerm / Ghostty / tmux	IDE yerine terminal kullan
Wispr Flow	Sesli prompt — 10x verimlilik iddiası
Wildcard permissions	<code>Bash(npm run *)</code> , <code>Edit(/docs/**)</code> — dangerously-skip yerine
<code>/sandbox</code>	Dosya ve ağ izolasyonu ile izin prompt'larını azalt
Agent Teams + tmux	Paralel geliştirme için çoklu agent
Git Worktrees	Her agent'a izole branch — paralel çalışma
Ralph Wiggum plugin	Otonom döngüde uzun görevleri tamamlama

10. Yapılması ve Yapılmaması Gerekenler

✓ YAP	✗ YAPMA
CLAUDE.md'yi 200 satır altında tut	Her şeyi tek CLAUDE.md'ye tıkma
Monorepo'da katmanlı CLAUDE.md kullan	Tek devasa CLAUDE.md ile yönetmeye çalışma
Feature-specific agent tanımla	Genel "QA engineer", "backend dev" agent kullanma
Workflow için command kullan	Basit iş akışı için agent açma
Her görev bitince hemen commit at	Uzun süre commit atmadan çalışma
%50'de <code>/compact</code> yap	Context dolana kadar bekleme
<code>Esc Esc</code> / <code>/rewind</code> ile geri sar	Aynı context'te hatayı düzeltmeye çalışma
Wildcard permission kullan	<code>dangerously-skip-permissions</code> kullanma
Claude Code'u her gün güncelle	Eski versiyonda takılı kalma
Plan mode ile başla	Direkt implementasyona dalma
Detaylı spec yaz, belirsizliği azalt	Muğlak prompt ile kaliteli çıktı bekleme
<code>settings.json</code> 'u git'e check-in et	Takım ayarlarını paylaşmadan çalışma
Agent'ı <code>Agent()</code> tool ile çağır	Agent'ı bash komutuyla çağırma
Sadece kullandığın MCP'leri ekle	15 MCP kurup "daha fazla = daha iyi" sanma
Vanilla CC'yi küçük işlerde tercih et	Her şey için workflow/agent kurma

Kaynak: shanraishan/claude-code-best-practice · Boris Cherny & Claude Code Ekibi · Anthropic Docs
Bu döküman referans rehberidir. Claude Code sürekli güncellenmektedir — resmi changelog'u takip edin.